

basis states $|0\rangle$ and $|1\rangle$, respectively. You will encounter these notations repeatedly throughout quantum computing, so I strongly recommend committing them to memory. They will soon become your close companions as we continue our journey into the quantum world.

A first look at the CNOT gate

Now, let us add a lot of power to the quantum computing model that we have been patiently developing by adding one more qubit. At first glance, this might look like a simple extension of the programs we have been writing for a single qubit. No! By moving from one qubit to two, we open the door to another fascinating quantum-mechanical phenomenon—entanglement. We have already encountered superposition while working with a single qubit. However, to create entanglement, we need at least two qubits that can interact with each other. Superposition, entanglement, interference, and other quantum-mechanical phenomena form the foundation on which quantum algorithms exploit the laws of quantum mechanics to tackle certain problems in ways that classical algorithms running on today's digital computers cannot.

It will take more than one article to fully appreciate the fascinating and sometimes rather tricky behaviour of entanglement. In fact, entanglement is arguably one of the most counterintuitive ideas in quantum mechanics. A famous quote often attributed to Niels Bohr goes, “Anyone who is not shocked by quantum theory has not understood it.” The exact wording and attribution of this quote are disputed, so let us not get caught up in that debate just yet! I am sure many of you are already wondering about entanglement and may have encountered some of the myths surrounding it, including the idea of ‘faster-than-light’ communication. Since entanglement is difficult enough to understand as a physical phenomenon, let us leave the

deeper mathematical and philosophical discussion for later. Instead, let us dive straight into a program. Consider the program *qiskit9.py*, shown below, which creates an entangled state involving two qubits using a CNOT gate. The CNOT gate is a two-qubit gate in which one qubit acts as the control and the other as the target. The target qubit is flipped from $|0\rangle$ to $|1\rangle$ or from $|1\rangle$ to $|0\rangle$ only when the control qubit is in the state $|1\rangle$; if the control qubit is $|0\rangle$, the target qubit remains unchanged. In our program, qubit 0 is the control and qubit 1 is the target.

```
1. from qiskit import QuantumCircuit, transpile
2. from qiskit_aer import AerSimulator
3. qc = QuantumCircuit(2, 2)
4. qc.h(0)
5. qc.cx(0, 1)
6. qc.measure([0, 1], [0, 1])
7. simulator = AerSimulator()
8. compiled = transpile(qc, simulator)
9. result = simulator.run(compiled, shots=1000).result()
10. counts = result.get_counts()
11. print(counts)
```

Now, let us try to understand the working of the program *qiskit9.py*. Line 1 imports the class *QuantumCircuit* and the function *transpile* from Qiskit. The function *transpile()* converts the circuit into a form suitable for the selected backend. Line 2 imports the class *AerSimulator* from the package *qiskit_aer*. The *AerSimulator* enables us to execute quantum circuits on a classical computer and observe their measurement outcomes. Line 3 creates a quantum circuit containing two qubits and two classical bits. Line 4 applies

```
(qiskit_env) # python qiskit9.py
{'11': 504, '00': 496}
(qiskit_env) # python qiskit9.py
{'00': 485, '11': 515}
(qiskit_env) # python qiskit9.py
{'00': 509, '11': 491}
(qiskit_env) # python qiskit9.py
{'11': 480, '00': 520}
```

Figure 5: Output of the program *qiskit9.py*

the Hadamard gate to qubit 0, putting it into an equal superposition of $|0\rangle$ and $|1\rangle$ (the $|+\rangle$ state). Line 5 applies the Controlled NOT (CNOT) gate, with qubit 0 as the control and qubit 1 as the target. Together, Lines 4 and 5 transform the initial state $|00\rangle$ into the entangled Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$. Line 6 measures both qubits and stores their measurement results in the corresponding classical bits. Thus, the measurement result of qubit 0 is stored in classical bit 0, while that of qubit 1 is stored in classical bit 1. Line 7 creates an instance of the *AerSimulator* and assigns it to the variable *simulator*. Line 8 transpiles the quantum circuit so that it is compatible with the selected simulator. Line 9 executes the transpiled circuit on the simulator for 1000 shots and stores the resulting data. Line 10 extracts the measurement statistics from the variable *result* and stores them in the variable *counts*. Finally, Line 11 prints these statistics on the screen. Since we are using an ideal simulator, the output will typically contain only 00 and 11, with approximately 500 occurrences of each. This striking correlation between the measurement outcomes of the two qubits is our first glimpse of the fascinating phenomenon of quantum entanglement.

Now, let us execute the program *qiskit9.py* and understand its output. Figure 5 shows the results of four successive executions of the program *qiskit9.py*. The measurement statistics obtained are $\{ '11': 504, '00': 496 \}$, $\{ '00': 485, '11': 515 \}$, $\{ '00': 509, '11': 491 \}$, and $\{ '11': 480, '00': 520 \}$, respectively. We know that when two independent qubits are measured, there are four possible outcomes: 00, 01, 10, and 11. However, the two qubits in our circuit are entangled and are therefore not independent of each other. If qubit 0 is measured as 0, qubit 1 will also be measured as 0 with certainty. Similarly, if qubit 0 is measured as 1, qubit 1 will also be measured as 1 with certainty.